

PEFT Methods Explained: LoRA, QLoRA, Prompt Tuning, Prefix Tuning, Adapters and IA³

Oct 9, 2026 · @Chirantan Lonkar

All six methods fine-tune a model by freezing its original numbers and training only a small set of new ones. They differ in where those new numbers sit and what they do. Every example uses TinyLlama (1.1 billion numbers), so the sizes can be compared directly.

Start here: the five ideas everything else builds on

1. A model is a large set of numbers

TinyLlama is a file of 1.1 billion numbers. They are called **parameters** or **weights**. Nothing else is stored in it: no sentences, no rules, only numbers.

2. Text travels through the model as lists of numbers

1. Your text is cut into pieces called **tokens**. `def add(a, b)` becomes `def`, `add`, `(`, `a`, `,`, `b`, `)`.
2. Each token is replaced by a **list of 2,048 numbers**. This list is the token's starting point inside the model.
3. The lists pass through **22 layers**, one after another. Each layer changes the numbers in every list.
4. After the last layer, the final list for the last token is turned into a score for each of the 32,000 possible next tokens. The highest score is the next word.

3. Inside a layer, the numbers sit in tables

Each layer holds several **tables** of numbers (also called matrices). A table changes a list by multiplying it. A small example with 3 numbers instead of 2,048:

	Value
Input list	[2, 1, 3]
Table row 1	[1, 0, 2] $\rightarrow 1 \times 2 + 0 \times 1 + 2 \times 3 = 8$
Table row 2	[0, 1, 1] $\rightarrow 0 \times 2 + 1 \times 1 + 1 \times 3 = 4$

	Value
Table row 3	$[3, 1, 0] \rightarrow 3 \times 2 + 1 \times 1 + 0 \times 3 = 7$
Output list	$[8, 4, 7]$

TinyLlama has 7 tables in each layer. The biggest are $2,048 \times 5,632$ numbers. Across all 22 layers, plus the word table at the start, they add up to the 1.1 billion parameters.

Each layer has two parts, in this order:

- **Attention part:** each token looks at the earlier tokens and decides which ones matter for it. To do this, each token produces a **key** list (used by others to decide how much to look at this token) and a **value** list (the information it passes on if looked at).
- **Feed-forward part:** each token's list is processed further on its own, through a wider middle list of 5,632 numbers.

4. Frozen numbers and trainable numbers

Training repeats one loop: show an example, measure how wrong the prediction was (the **loss**), then nudge numbers slightly to make it less wrong.

- A **trainable** number is one training is allowed to nudge.
- A **frozen** number is never changed. The model still uses it in every calculation; training just leaves it alone.
- "**Trainable parameters**" = the count of numbers training may change. When a method says "12.6 million trainable", every other number in the model is frozen.

5. What a "trainable factor" is

A **factor** is a number that multiplies another number. A **trainable factor** is a multiplier whose value training is allowed to change.

Original number	Factor	Result	Effect
0.8	1.0	0.8	unchanged
0.8	1.5	1.2	made stronger
0.8	0.5	0.4	made weaker
0.8	0.0	0.0	switched off

IA³ (the last method) is built entirely from trainable factors. The other methods train other kinds of numbers: small tables, made-up token lists, or small extra layers.

Why training all 1.1 billion numbers is a problem

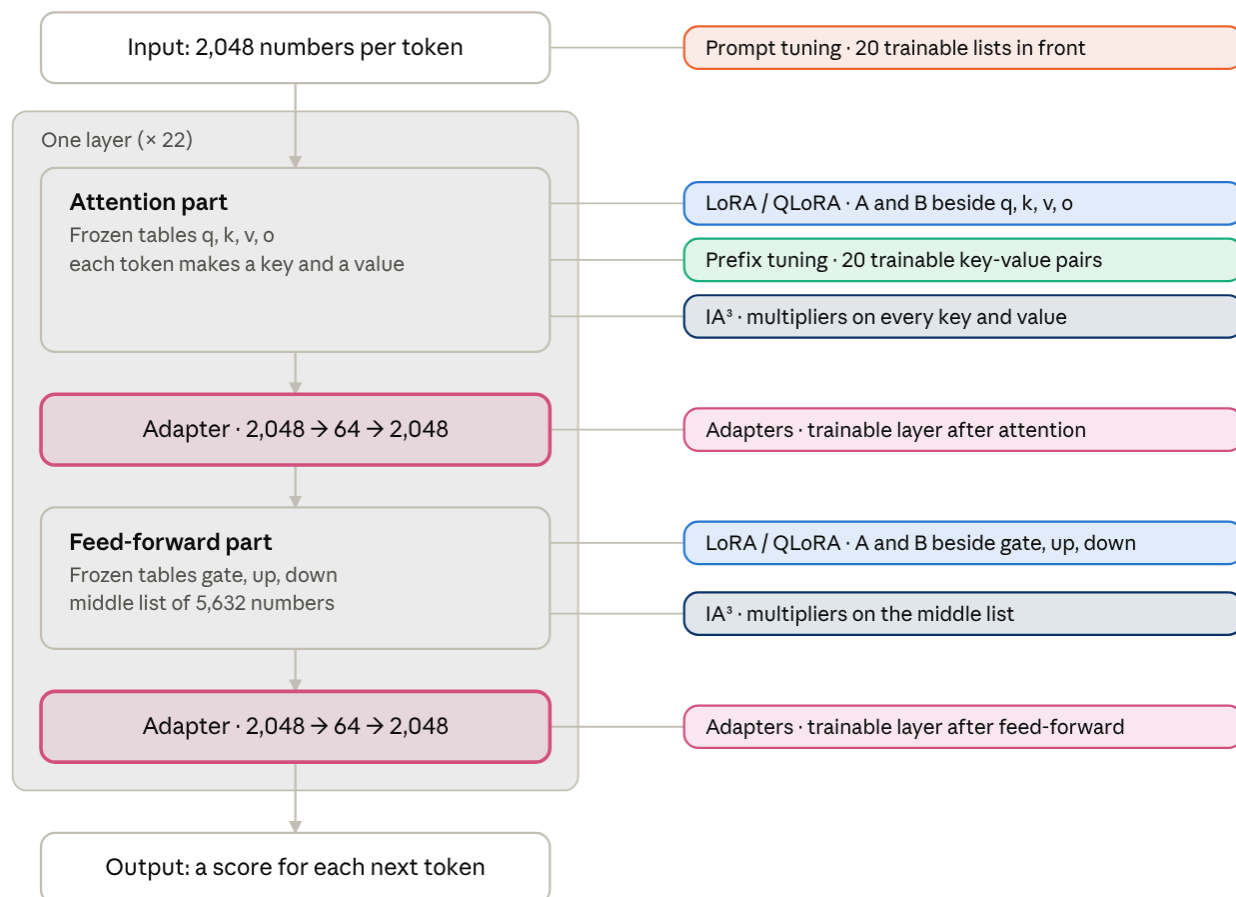
Item	Full fine-tuning of TinyLlama
Numbers training must keep per trainable number	about 4: the number, how much to nudge it, and 2 tracking values
Memory for that	1.1 billion $\times$ 4 numbers $\times$ 4 bytes $\approx$ 17.6 GB
Memory on a free Colab T4	about 15 GB, so it does not fit
What you save at the end	a whole new 2.2 GB copy of the model, for one task

Every method below solves this the same way: freeze all 1.1 billion numbers, and train only a small new set. That family of methods is called **PEFT** (Parameter-Efficient Fine-Tuning).

Where each method plugs in

All six freeze the model; they differ in where new numbers attach

One token's path through TinyLlama. Grey outline = frozen; coloured = trained.



where each method attaches · one TinyLlama layer, repeated 22 times

Read it top to bottom: a token's list enters, passes through 22 identical layers, and comes out as next-token scores. Grey boxes never change; each coloured tag is the only part its method trains. QLoRA attaches exactly where LoRA does; the difference is that the grey tables are stored in 4 bits.

LoRA (Low-Rank Adaptation)

LoRA freezes every original table and places two small trainable tables, **A** and **B**, beside it; their product is a correction added to the table's output. In TinyLlama this trains 12.6 million numbers, 1.1% of the model.

The core idea: new table = old table + change table

Any fine-tuning ends with "new table = old table + change table". Full fine-tuning learns every number of the change table one by one: $2,048 \times 2,048 = 4,194,304$ numbers for one table. LoRA builds the same-sized change table from **two short lists** multiplied together.

Worked by hand: 8 numbers make a 16-number table

Take a 4×4 table (16 numbers). LoRA keeps two lists of 4 numbers each:

- List B (going down): 1, 2, 0, -1
- List A (going across): 0.1, 0, 0.2, -0.1

Rule: multiply each number in B by the whole of A. Each result is one row of the change table.

B value	× A = row of the change table
1	[0.1, 0, 0.2, -0.1]
2	[0.2, 0, 0.4, -0.2]
0	[0, 0, 0, 0]
-1	[-0.1, 0, -0.2, 0.1]

8 trained numbers produced a full 16-number change table. The original table was never edited.

What "low rank" means

All 4 rows above are the same pattern, [0.1, 0, 0.2, -0.1], multiplied by 1, 2, 0 or -1.

- **Rank** = how many different patterns a table is built from. The table above has rank 1.

- A fully free 4×4 table could have 4 unrelated rows (rank 4) and needs all 16 numbers.
- **Low rank** = the change table may only use a few patterns, so it needs few numbers.

The LoRA authors found that the change needed for one new task can be built from few patterns. With rank $r = 16$, each change table mixes 16 patterns: A becomes $16 \times 2,048$ and B becomes $2,048 \times 16$.

What happens to one token, step by step

Using the 3-number example from "Start here" (input $[2, 1, 3]$, original output $[8, 4, 7]$), with a rank-1 LoRA where $A = [1, 0, 1]$ and $B = [0.2, 0, -0.4]$:

1. **Original path (frozen):** the table turns $[2, 1, 3]$ into $[8, 4, 7]$, exactly as before.
2. **Squeeze with A:** $1 \times 2 + 0 \times 1 + 1 \times 3 = 5$. The whole input becomes one number (with $r = 16$, it becomes 16 numbers).
3. **Expand with B:** $5 \times [0.2, 0, -0.4] = [1, 0, -2]$. This is the correction.
4. **Add:** $[8, 4, 7] + [1, 0, -2] = [9, 4, 5]$. That is the layer's new output.

The correction is also multiplied by a fixed scale, `lora_alpha / r`. Our notebooks use $\alpha = 32$ and $r = 16$, so the scale is 2.

Training, from first step to last

1. **Freeze** all 1.1 billion original numbers.
2. **Attach** an A and B pair beside each chosen table. TinyLlama: 7 tables $\times$ 22 layers = 154 pairs.
3. **Start with B = all zeros.** The correction is then zero, so the model behaves exactly like the original at step 0. A starts with small random numbers.
4. **For each batch:** predict, measure the loss, nudge only the numbers in A and B.
5. **After training,** B is no longer zero, so every table now adds a learned correction.

Where TinyLlama's 12.6 million trainable numbers come from

Each pair adds $16 \times (\text{rows} + \text{columns})$ numbers:

Table in each layer	Size	LoRA numbers ($r = 16$)
<code>q_proj</code> , <code>o_proj</code> (attention)	$2,048 \times 2,048$	65,536 each
<code>k_proj</code> , <code>v_proj</code> (attention)	$2,048 \times 256$	36,864 each
<code>gate_proj</code> , <code>up_proj</code> , <code>down_proj</code> (feed-forward)	$2,048 \times 5,632$	122,880 each
One layer		573,440

Table in each layer	Size	LoRA numbers (r = 16)
22 layers		12,615,680 (1.13%)

In your TinyLlama run, `print_trainable_parameters()` printed exactly 12,615,680.

The LoRA adapter and merging

- **LoRA adapter** = the saved A and B tables plus a small settings file. Your TinyLlama adapter was 48 MB; the base model is 2.2 GB and is not saved again.
- **Swapping**: one base model can load a Python adapter today and a SQL adapter tomorrow.
- **Merging**: before deployment, the change table $B \times A$ can be added permanently into the original table. The result is one normal model that answers exactly as fast as the original.

What happens and what does not

Happens	Does not happen
A and B are trained beside 154 tables	No original number changes
Each table's output gets a learned correction added	No new layers are inserted in the path
The adapter is saved separately (tens of MB)	The base model is not copied

The settings you choose

Setting	Our value	Meaning
<code>r</code>	16	patterns per change table. Bigger = more trainable numbers
<code>lora_alpha</code>	32	sets the scale $\alpha / r = 2$
<code>target_modules</code>	7 table names	which tables get an A and B pair
<code>lora_dropout</code>	0.05	during training, randomly switch off 5% of LoRA inputs to discourage memorising

Good: high quality, the most widely used method, mergeable, small files. **Limit**: the frozen model still needs its full memory, which QLoRA fixes.

QLoRA (Quantized LoRA)

QLoRA is LoRA on top of a model whose frozen numbers are stored in 4 bits instead of 32. The trainable part is exactly the same as LoRA; only the frozen model shrinks. In your TinyLlama run the model used **0.70 GB** of GPU memory instead of about 4.1 GB.

Step 1: why 4 bits allows only 16 values

A bit is 0 or 1. Four bits can only form these 16 patterns:

```
0000 0001 0010 0011 0100 0101 0110 0111
1000 1001 1010 1011 1100 1101 1110 1111
```

So each stored weight must be one of 16 allowed levels. A 32-bit number can be about 4 billion different values; a 4-bit number only 16.

Step 2: worked by hand, compressing 5 weights

Weights are compressed in groups. Here is a group of 5 (real groups have 64): 0.037, -0.012, 0.081, -0.064, 0.005.

1. **Find the scale:** the largest value ignoring the sign is **0.081**. It is stored once for the group.
2. **Divide every weight by 0.081**, so all fall between -1 and +1.
3. **Round each to the nearest of the 16 levels** and store only its 4-bit code.
4. **When the GPU needs a weight**, it multiplies the level by 0.081 again.

Original	÷ 0.081	Nearest level	Stored code	Used as (level × 0.081)
0.037	0.457	0.467	1011	0.0378
-0.012	-0.148	-0.200	0110	-0.0162
0.081	1.000	1.000	1111	0.0810
-0.064	-0.790	-0.733	0010	-0.0594
0.005	0.062	0.067	1000	0.0054

The last column is close to the first, not exact. That small loss of precision buys 8 times less storage. (For simplicity the levels here are evenly spaced; the real scheme spaces them differently, see below.)

Step 3: put LoRA on top

1. Load the model and compress every big table to 4 bits while loading.
2. Freeze it. The 4-bit numbers are **never trained**: rounding to 16 levels leaves no room for small nudges.
3. Attach the LoRA A and B tables in **normal precision** (32-bit), because they are the part being nudged.
4. During each calculation, the GPU expands the needed 4-bit weights to 16-bit, multiplies, then discards the expanded copy.

The three QLoRA settings, in plain words

Setting	Meaning
<code>bnb_4bit_quant_type="nf4"</code>	NF4 (NormalFloat 4) places more of the 16 levels near zero, because most weights are close to zero. Small weights get rounded more precisely
<code>bnb_4bit_use_double_quant=True</code>	Each group of 64 weights keeps one scale number. Double quantization compresses those scale numbers too, saving a little more memory
<code>bnb_4bit_compute_dtype=torch.float16</code>	Calculations happen in 16-bit, which the T4 handles quickly; storage stays 4-bit

Memory for TinyLlama

Stored as	Memory for the model
32-bit (full precision)	about 4.1 GB
16-bit (as downloaded)	about 2.2 GB
4-bit with QLoRA (measured in your run)	0.70 GB

It is not exactly one-eighth of 4.1 GB because the word table at the start and the word scores at the end stay in 16-bit, and the scale numbers take some space.

What happens and what does not

Happens	Does not happen
Frozen tables are stored in 4 bits	The 4-bit numbers are never trained
LoRA A and B are trained in 32-bit	The trainable count does not change: still 12.6 million
Weights are expanded to 16-bit only while calculating	The saved adapter does not contain the 4-bit model

Good: fine-tunes much larger models on small GPUs; both our notebooks use it. **Limits:** rounding costs a little accuracy, and training is somewhat slower because weights are expanded on every use. A 1B model is more sensitive to 4-bit rounding than a 70B model, so always compare answers on your own test questions.

Prompt tuning

Prompt tuning freezes the entire model and trains only a few made-up "tokens" placed in front of every input. With 20 of them, TinyLlama trains **40,960** numbers, about 0.004% of the model.

First, a normal input

Every real token becomes a list of numbers (2,048 in TinyLlama; 3 here so it fits on paper). These lists come from the model's word table, which is frozen.

Token	Its list (made-up values)
Reverse	[0.2, -0.1, 0.5]
a	[0.0, 0.3, -0.2]
string	[0.4, 0.1, 0.1]

What prompt tuning adds

It puts extra lists **in front** of the real ones. They are not words: they do not exist in the 32,000-token word list and have no text. They are just lists of numbers that training is allowed to change. This example uses 2; a typical setting is 20.

Position	Token	Its list	Trained?
1	P1 (made-up)	[0.7, -0.3, 0.2]	Yes
2	P2 (made-up)	[-0.1, 0.6, 0.0]	Yes
3	Reverse	[0.2, -0.1, 0.5]	No
4	a	[0.0, 0.3, -0.2]	No
5	string	[0.4, 0.1, 0.1]	No

What happens, step by step

1. The input is built as [P1 ... P20] + [the user's real tokens].
2. The frozen model processes all of them as usual. In every layer's attention part, the real tokens can look at P1 ... P20, so those lists influence every later calculation.
3. The loss is measured on the answer, as always.
4. Training works out how each number **in P1 ... P20 only** should change to lower the loss, and nudges them. Every model number stays frozen.
5. Over many steps, P1 ... P20 become lists that push the model towards the desired behaviour, for example "answer with Python code only".

How it differs from writing a prompt yourself

	A written prompt	Prompt tuning (a "soft prompt")
Made of	real words, chosen by a person	free lists of numbers, chosen by training
Possible values	only the 32,000 existing token lists	any numbers at all
Readable	yes	no: they match no word
Improves with examples	only if a person rewrites it	yes, by training

A common starting point is to copy the lists of real words, such as "Answer with Python code only", into P1 ... P20, then let training adjust them from there.

The numbers

- Trainable: $20 \times 2,048 = \mathbf{40,960}$ numbers.

- Saved file: about 160 KB (40,960 numbers × 4 bytes), compared with 48 MB for your LoRA adapter.

What happens and what does not

Happens	Does not happen
20 learned lists are added before every input	No table, layer or word list inside the model changes
Every layer sees them through attention	Nothing is added inside the layers themselves
One frozen model can serve many tasks by swapping the 20 lists	The learned lists cannot be merged into the model; they must be added to every request

Good: the smallest file of all, and many tasks can share one model. **Limits:** the research that introduced it (Lester and colleagues, 2021) found it matches full fine-tuning mainly on very large models, around 10 billion parameters and up; on a 1B model like TinyLlama it is usually clearly weaker than LoRA. It also uses up 20 positions of every input.

Prefix tuning

Prefix tuning freezes the model and adds learned items directly inside the attention part of **every layer**, not only at the input. With 20 items, TinyLlama trains **225,280** numbers, about 0.02% of the model.

First, how attention uses keys and values

In every layer, each token produces three lists from its numbers:

- a **query**: what this token is looking for in earlier tokens;
- a **key**: what this token offers, used by later tokens to decide whether to look at it;
- a **value**: the information this token hands over when looked at.

To update the token "string" in one layer:

1. Compare the query of "string" with the key of every earlier token. A closer match gives a higher score.
2. Turn the scores into shares that add up to 100%, for example "Reverse" 70%, "a" 10%, "string" 20%.
3. Take a mix of the value lists in those shares. That mix becomes the update for "string".

What prefix tuning adds

In every layer, it adds **20 extra (key, value) pairs** that come from no token at all. They are learned numbers, placed in front of the real tokens' keys and values. One layer, with 2 extra pairs:

Item in this layer	Key	Value	Trained?
Prefix 1	learned	learned	Yes
Prefix 2	learned	learned	Yes
Reverse	made by the frozen layer	made by the frozen layer	No
a	made by the frozen layer	made by the frozen layer	No
string	made by the frozen layer	made by the frozen layer	No

When "string" looks back, it now compares its query with 5 keys instead of 3. If a prefix key matches well, part of the update for "string" comes from that prefix's learned value. That is how the prefix steers every token, in every layer.

What happens, step by step

1. Freeze the whole model.
2. In each of the 22 layers, create 20 learned key lists and 20 learned value lists.
3. Run each example. In every layer, every real token can attend to those 20 pairs as well as to the real earlier tokens.
4. Measure the loss on the answer and nudge **only the prefix keys and values**.

The original method (Li and Liang, 2021) trains the prefix through a small extra network during training, because adjusting the pairs directly was unstable. That helper network is thrown away afterwards; only the final pairs are kept.

How it differs from prompt tuning

	Prompt tuning	Prefix tuning
Where the learned numbers sit	only at the input, before layer 1	inside attention in all 22 layers
What they are	token-like lists of 2,048	key and value lists for each layer

	Prompt tuning	Prefix tuning
How they reach later layers	indirectly: frozen layers transform them first	directly: each layer has its own learned pairs
Trainable numbers (20 items, TinyLlama)	40,960	225,280

The numbers

TinyLlama's keys and values are 256 numbers each, so: $20 \text{ items} \times 22 \text{ layers} \times 2 \text{ (key and value)} \times 256 = \mathbf{225,280}$ trainable numbers.

What happens and what does not

Happens	Does not happen
Every layer gets 20 learned key-value pairs	No table inside the model changes
Every real token can attend to them	No new layers are added
Only the pairs are saved (under 1 MB)	They cannot be merged into the model; they are added to every request

Good: still tiny, and stronger than prompt tuning because it acts in every layer. **Limits:** usually behind LoRA in quality, training can be unstable, and the 20 extra items take up attention space in every request.

Adapters

Adapters freeze the model and insert small new layers inside each existing layer: one after the attention part and one after the feed-forward part. In TinyLlama that is 44 adapters and about **11.5 million** trainable numbers, about 1% of the model.

What one adapter is

An adapter is a tiny layer with three steps, plus an add-back:

1. **Shrink:** a small table turns the list of 2,048 numbers into a short list of 64.
2. **Switch off negatives:** every negative number in the short list becomes 0. This in-between step lets the adapter learn patterns a single multiplication cannot.
3. **Expand:** a second small table turns the 64 numbers back into 2,048.

4. **Add back:** the result is added to the list that came in. So the adapter only ever adds a correction; it never replaces what passed through.

Worked by hand: 4 numbers $\rightarrow 2 \rightarrow 4$

Input list: [2, 1, 3, 0]

Shrink with two rows:

Shrink row	Calculation	Result
[0.5, 0, 0.5, 0]	$0.5 \times 2 + 0 \times 1 + 0.5 \times 3 + 0 \times 0$	2.5
[0, -1, 0, 1]	$0 \times 2 - 1 \times 1 + 0 \times 3 + 1 \times 0$	-1

Switch off negatives: [2.5, -1] becomes [2.5, 0].

Expand with four rows:

Expand row	Calculation	Result
[0.2, 0.0]	$0.2 \times 2.5 + 0 \times 0$	0.5
[0.0, 1.0]	$0 \times 2.5 + 1 \times 0$	0
[-0.4, 0.3]	$-0.4 \times 2.5 + 0.3 \times 0$	-1.0
[0.1, 0.0]	$0.1 \times 2.5 + 0 \times 0$	0.25

Add back: [2, 1, 3, 0] + [0.5, 0, -1.0, 0.25] = [2.5, 1, 2.0, 0.25]. That goes on to the rest of the layer.

Training, step by step

1. Freeze the whole model.
2. Insert 2 adapters in each of the 22 layers: 44 in total.
3. **Start the expand tables near zero**, so each adapter adds almost nothing and the model behaves like the original at step 0.
4. For each batch: predict, measure the loss, nudge only the adapter tables.

The numbers

Each adapter: $2,048 \times 64$ (shrink) + $64 \times 2,048$ (expand) = 262,144 numbers, plus about 2,000 small extra numbers. 44 adapters $\approx$ **11.5 million** trainable numbers.

Why adapters slow down answering, but LoRA does not

	LoRA	Adapters
Where it sits	beside an existing table	in line, after a part of the layer
Contains a "switch off negatives" step	no: only multiplications	yes
Can be folded into the original tables	yes: $B \times A$ is added into the table once	no: the in-between step cannot be folded into a table
Extra work when answering	none after merging	every token passes through 44 extra small layers

What happens and what does not

Happens	Does not happen
44 small layers are inserted into the model's path	No original table changes
Each adds a learned correction to what passes through	They cannot be merged away
Only the adapters are saved	The base model is not copied

Good: quality close to LoRA, and the oldest PEFT method (Houlsby and colleagues, 2019).

Limits: answers are slightly slower forever, because the extra layers always run. They are not part of the core `peft` library; the separate `adapters` library (AdapterHub) provides them. Do not confuse them with a **LoRA adapter**, which is the saved file of LoRA tables.

IA³

IA³ freezes the model and trains only **multipliers**: one trainable factor per number at three places inside each layer, which make those numbers stronger, weaker or switched off.

TinyLlama gets **135,168** trainable factors, about 0.01% of the model.

The name stands for "Infused Adapter by Inhibiting and Amplifying Inner Activations" (Liu and colleagues, 2022). The three I's and the A give "IA³". *Inhibiting* = weakening, *amplifying* = strengthening, *inner activations* = the lists of numbers produced inside the model.

Where the factors sit

In each layer, IA³ attaches a list of factors to three lists the model produces:

Place in the layer	Length of that list in TinyLlama	Factors added
Attention keys	256	256
Attention values	256	256
Feed-forward middle list	5,632	5,632
One layer		6,144
22 layers		135,168

Worked by hand: multiplying number by number

Suppose a value list has 4 numbers (256 in reality). Each position has its own trainable factor:

Position	Value from the frozen model	Trainable factor	Result	Effect
1	0.8	1.0	0.8	unchanged
2	-1.2	1.5	-1.8	strengthened
3	0.5	0.0	0.0	switched off
4	2.0	0.5	1.0	weakened

The **same 4 factors** are applied to the value list of every token. They do not depend on which word it is: position 3 is switched off for every token in this layer.

Training, step by step

1. Freeze the whole model.
2. Attach a factor list at the keys, the values and the feed-forward middle of each layer.
3. **Start every factor at 1.0**, so nothing changes and the model behaves exactly like the original at step 0.
4. For each batch: predict, measure the loss, nudge only the factors. A factor drifting above 1 strengthens that position; below 1 weakens it.

Merging

Multiplying a list by fixed factors gives the same result as scaling the matching rows or columns of a neighbouring table: the table that produced the list, or the one that uses it next. So after training, the factors can be folded into those tables once, and the model answers as fast as the original.

What happens and what does not

Happens	Does not happen
Existing internal numbers are scaled up, down or to zero	Nothing new is created: no new tables, tokens or layers
One factor per position, shared by every token	The factors do not change from word to word
Factors can be merged into the tables	No extra work when answering, after merging

Good: among the smallest of all methods, quick to train, and mergeable. **Limits:** it can only strengthen or weaken what the model already produces; it cannot build new combinations the way LoRA's A and B can. So on harder tasks it is usually weaker than LoRA.

All six side by side (TinyLlama, 1.1 billion numbers)

LoRA and QLoRA give the best quality for the effort; prompt tuning, prefix tuning and IA³ are far smaller but weaker on small models; adapters match LoRA but slow every answer.

Method	Where it acts	What is trained	Starts as	Trainable numbers	Saved file	Slower when answering?	Quality on a 1B model
LoRA	beside 7 tables in every layer	small A and B tables ($r = 16$)	$B = 0$, so no change	12.6 M (1.1%)	48 MB	no, once merged	high
QLoRA	same as LoRA; model stored in 4-bit	same A and B tables	$B = 0$	12.6 M (1.1%)	48 MB	no, once merged	high, with 0.7 GB instead of 4.1 GB of model memory
Prompt tuning	in front of the input only	20 made-up token lists	random or copied from real words	41 K (0.004%)	160 KB	slightly: 20 extra input positions	low
Prefix tuning	inside attention in every layer	20 key-value pairs per layer	random, via a helper network	225 K (0.02%)	0.9 MB	slightly: 20 extra items per layer	medium
Adapters	small layers inserted after attention and feed-forward	44 shrink-and-expand layers	expand near 0, so no change	11.5 M (1%)	46 MB	yes: 44 extra layers always run	high

Method	Where it acts	What is trained	Starts as	Trainable numbers	Saved file	Slower when answering?	Quality on a 1B model
IA ³	keys, values and feed-forward middle of every layer	one multiplier per number	all 1.0, so no change	135 K (0.01%)	0.5 MB	no, once merged	medium

Every method except prompt tuning and prefix tuning starts as "no change" and lets training move away from the original model gradually; all six leave the 1.1 billion original numbers untouched. The LoRA and QLoRA numbers are from your TinyLlama run; the others are calculated from TinyLlama's sizes with typical settings (20 items, a shrink to 64).

Which one to use, and the code for each

Default to LoRA, or QLoRA when GPU memory is tight; reach for the others only for a specific reason below.

Your situation	Use	Why
Most fine-tuning tasks, enough GPU memory	LoRA	best quality for the effort, mergeable
Model barely fits, or a free T4	QLoRA	same quality idea, model memory cut about 6 times
A very large model (10B+) serving many tasks, smallest possible files	Prompt tuning or IA ³	files under 1 MB per task
Need more control than prompt tuning, still tiny	Prefix tuning	acts in every layer
Existing project already built on them	Adapters	similar quality to LoRA, but slower answers

The code: same two lines, different config

Every method is applied the same way in the `peft` library: write a config, then `model = get_peft_model(model, config)`. Only the config changes.

```
from peft import LoraConfig, PromptTuningConfig, PrefixTuningConfig,
IA3Config, get_peft_model

# LoRA: A and B tables beside 7 tables per layer
```

```

config = LoraConfig(task_type="CAUSAL_LM", r=16, lora_alpha=32,
                    lora_dropout=0.05,
                    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                                    "gate_proj", "up_proj", "down_proj"])

# QLoRA: the same LoraConfig, on a model loaded with
BitsAndBytesConfig(load_in_4bit=True, ...)

# Prompt tuning: 20 made-up tokens, started from the lists of real words
config = PromptTuningConfig(task_type="CAUSAL_LM", num_virtual_tokens=20,
                            prompt_tuning_init="TEXT",
                            prompt_tuning_init_text="Answer with Python code only",
                            tokenizer_name_or_path=MODEL_ID)

# Prefix tuning: 20 key-value pairs in every layer
config = PrefixTuningConfig(task_type="CAUSAL_LM", num_virtual_tokens=20)

# IA3: multipliers on keys, values and the feed-forward middle (the input of
down_proj)
config = IA3Config(task_type="CAUSAL_LM", target_modules=["k_proj", "v_proj",
                                                           "down_proj"],
                  feedforward_modules=["down_proj"])

model = get_peft_model(model, config)
model.print_trainable_parameters()

```

Classic adapters are not in the core `peft` library; the separate `adapters` library (AdapterHub) provides them.

Check yourself

- 1. In LoRA, why does the model behave exactly like the original before training starts?**
Because B starts as all zeros, so $B \times A$ adds nothing to any table's output.
- 2. In QLoRA, which numbers are trained: the 4-bit ones or the 32-bit ones?** Only the 32-bit LoRA tables. The 4-bit numbers are frozen; rounded to 16 levels, they cannot take small nudges.
- 3. What is the difference between prompt tuning and prefix tuning?** Prompt tuning adds learned lists only before the input. Prefix tuning adds learned key-value pairs inside attention in every layer, so it acts directly everywhere.
- 4. Why do adapters slow answers but LoRA does not?** Adapters sit in the model's path and contain a "switch off negatives" step, so they cannot be folded into the tables and always run. LoRA's $B \times A$ can be added into the tables once.
- 5. What is a trainable factor in IA³?** A multiplier that starts at 1.0 and that training may move: above 1 strengthens a number, below 1 weakens it, 0 switches it off.

6. **Is a "LoRA adapter" the same as the "Adapters" method?** No. A LoRA adapter is the saved file of LoRA tables. Adapters is a different method that inserts small extra layers.
7. **All six methods share one rule. What is it?** Freeze every original number, and train only a small new set.